

Master Summary: Hierarchical k -GNN Explanations via GIN-Graph

Bachelor Thesis Defence Preparation Document

A complete walk-through of sections 1–4 of the thesis, organized for defence prep. The goal: be able to derive any forward pass, any backward pass, any gradient, and any score from scratch on a whiteboard, and clearly state the two technical contributions.

The document is split into three parts:

- **Part I: The classifier** (f_θ) — 1-GNN and 1-2-GNN forward and backward pass, training loop. *Sections 3.1–3.2 of the thesis.*
- **Part II: The generator** (G_ϕ via GIN-Graph) — generator architecture, loss functions, training dynamics, dense wrapper. *Sections 3.3–3.5.*
- **Part III: The validation score** — how trained generators are evaluated. *Section 3.6.*

Two technical contributions to flag throughout:

1. **The dense wrapper for the hierarchical 1-2-GNN** (Part II, section 7).
2. **The corrected validation metric** using actual cosine similarity to a class centroid (Part III).

Contents

Part I — The Classifier: 1-GNN and 1-2-GNN	2
1 Notation	2
2 Forward pass	2
2.1 1-GNN block (operates on nodes)	2
2.2 Bridge: build pair features (Eq. 8 in the thesis)	3
2.3 2-GNN block (operates on pairs)	3
2.4 Two readouts and concatenation	3
2.5 Classifier MLP and softmax	3
2.6 Cross-entropy loss	3
3 The whole forward pass as one function	3
4 Backward pass	4
4.1 Logits $\rightarrow$ loss	4
4.2 Through the MLP	4
4.3 Split at the concatenation	4

4.4	Pair branch: through the 2-GNN	4
4.5	Through the bridge concatenation	4
4.6	Node branch and the merge at $H^{(2)}$	5
4.7	Through the 1-GNN block	5
5	Weight update	5
6	What you've built	5
7	Two facts to memorize for the defence	6
Part II — The Generator: GIN-Graph Training Pipeline		7
8	The big picture	7
9	Generator architecture	7
9.1	Stage 1: MLP backbone (Eqs 13–15)	7
9.2	Stage 2: Gumbel-Softmax sampling (Eqs 16–18)	7
10	The full generator loss	8
10.1	Where the class enters each component	8
11	Component 1: $\mathcal{L}_{\text{reg}}$ (structural regularization)	8
11.1	Size penalty	8
11.2	Node-type penalty	9
11.3	Gradient path	9
12	Component 2: $\mathcal{L}_{\text{GAN}}$ (adversarial loss)	9
12.1	Gradient path	9
13	Component 3: $\mathcal{L}_{\text{GNN}}$ (classifier-guidance loss)	10
13.1	The full chain rule	10
13.2	Gradient path (visualised)	10
14	The dense wrapper (section 3.5) — <i>the technical contribution</i>	10
14.1	The problem	10
14.2	The fix	11
14.3	Dense 1-GNN forward (Eq 25)	11
14.4	Dense 1-2-GNN — the actual contribution	11
14.5	Where $\tilde{A}$ enters the autograd graph in the 1-2-GNN dense wrapper	12
14.6	Defence answer in one paragraph	12
15	Dynamic weight $\lambda(t)$ (curriculum learning, Eq 23)	12
16	The full training loop	12
16.1	Step 1: Update discriminator ψ	12
16.2	Step 2: Update generator ϕ	13
16.3	The merged gradient picture	13
16.4	Repeat for ~ 300 epochs	13
17	After training: generating explanations	13

Part III — The Validation Score	14
18 The composite score (Eq 31)	14
19 Component 1: prediction probability p	14
20 Component 2: embedding similarity s — <i>the corrected one</i>	14
20.1 The “corrected” part — second contribution of the thesis	15
21 Component 3: degree score d	15
22 Granularity κ (Eq 34, separate from v)	15
23 How scores get used in section 5	15
 Defence Prep Checklist	 17
24 The two technical contributions	17
25 The two structural facts to internalize	17
26 The clean defence one-liners	17

Part I — The Classifier: 1-GNN and 1-2-GNN

A complete walk-through of one training step on a single graph (A, X) with true label $y \in \{1, \dots, C\}$. I use $T_1 = 2$ message-passing layers in the 1-GNN block and $T_2 = 1$ in the 2-GNN block to keep indices readable; more layers is the same recipe repeated.

1 Notation

Symbol	Meaning
n	number of nodes in the graph
$P = \binom{n}{2}$	number of unordered node pairs
d	input feature dimension (e.g. 7 atom types for MUTAG)
d_T	hidden dimension (= 64 in the thesis)
d_h	classifier MLP hidden width
C	number of classes
$X \in \mathbb{R}^{n \times d}$	input node feature matrix
$A \in \{0, 1\}^{n \times n}$	(symmetric) adjacency matrix of G
$\mathcal{A} \in \{0, 1\}^{P \times P}$	pair-adjacency, $\mathcal{A}_{ss'} = 1$ iff s' is a swap-one neighbour of s
$H^{(t)} \in \mathbb{R}^{n \times d_T}$	node embeddings after 1-GNN layer t
$F^{(\ell)} \in \mathbb{R}^{P \times d_T}$	pair embeddings after 2-GNN layer ℓ
$W_1^{(t)}, W_2^{(t)}$	1-GNN weights at layer t (self / neighbour)
$W_3^{(\ell)}, W_4^{(\ell)}$	2-GNN weights at layer ℓ (self / neighbour)
W_c, W_o	classifier MLP weights
$\sigma(\cdot)$	ReLU, applied elementwise; $\sigma'(x) = \mathbb{I}[x > 0]$
$\odot$	elementwise (Hadamard) product
$\parallel$	concatenation
$\mathbf{1}_n$	all-ones column vector of length n

The full set of trainable parameters is

$$\theta = \{W_1^{(1)}, W_2^{(1)}, W_1^{(2)}, W_2^{(2)}, W_3^{(1)}, W_4^{(1)}, W_c, W_o\}$$

2 Forward pass

2.1 1-GNN block (operates on nodes)

Layer 1. Each node updates by combining its own previous feature (via W_1) with the sum of its graph-neighbours' previous features (via W_2):

$$Z^{(1)} = XW_1^{(1)} + AXW_2^{(1)}, \quad H^{(1)} = \sigma(Z^{(1)})$$

I separate the **pre-activation** $Z^{(1)}$ from the **post-activation** $H^{(1)}$ because the ReLU derivative $\mathbb{I}[Z^{(1)} > 0]$ shows up in the backward pass.

Layer 2.

$$Z^{(2)} = H^{(1)}W_1^{(2)} + AH^{(1)}W_2^{(2)}, \quad H^{(2)} = \sigma(Z^{(2)})$$

After this, $H^{(2)} \in \mathbb{R}^{n \times d_T}$ holds the final node embeddings — each row $H_{v,\cdot}^{(2)}$ is node v 's vector summarizing its 2-hop neighbourhood.

2.2 Bridge: build pair features (Eq. 8 in the thesis)

For every unordered pair $\{u, v\}$ with $u < v$:

$$F_{\{u,v\}}^{(0)} = [H_{u,:}^{(2)} \parallel H_{v,:}^{(2)} \parallel A_{uv}] \in \mathbb{R}^{2d_T+1}$$

The three pieces are: the embeddings of the two endpoints, plus the **isomorphism-type bit** A_{uv} telling the 2-GNN whether this pair is bonded in G . Concatenation is differentiable: its gradient just splits the incoming signal back into the pieces it was built from.

2.3 2-GNN block (operates on pairs)

One layer of swap-based message passing on the pair-graph:

$$\mathcal{Z}^{(1)} = F^{(0)}W_3^{(1)} + \mathcal{A}F^{(0)}W_4^{(1)}, \quad F^{(1)} = \sigma(\mathcal{Z}^{(1)})$$

Structurally identical to a 1-GNN layer — just with $\mathcal{A}$ instead of A , F instead of H , and pair-level weights.

2.4 Two readouts and concatenation

$$h_G^{(1)} = \mathbf{1}_n^\top H^{(2)} \in \mathbb{R}^{d_T}, \quad h_G^{(2)} = \mathbf{1}_P^\top F^{(1)} \in \mathbb{R}^{d_T}$$

$$h_G = [h_G^{(1)} \parallel h_G^{(2)}] \in \mathbb{R}^{2d_T}$$

Concatenation rather than addition: addition would force both branches into the same vector space and lose the distinction. Concatenation preserves both signals as separate slots.

2.5 Classifier MLP and softmax

$$a = W_ch_G \in \mathbb{R}^{d_h}, \quad b = \sigma(a), \quad z = W_ob \in \mathbb{R}^C$$

The vector z holds the **logits**. Softmax turns them into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

2.6 Cross-entropy loss

$$\mathcal{L} = -\log p_y = -z_y + \log \sum_{j=1}^C e^{z_j}$$

This second form (negative correct logit + log-sum-exp) is the one to differentiate.

3 The whole forward pass as one function

Substituting everything backwards, the loss is one giant scalar function of θ :

$$\mathcal{L}(\theta; A, X, y) = -\left[W_o \sigma\left(W_c [\mathbf{1}_n^\top H^{(2)} \parallel \mathbf{1}_P^\top \sigma(F^{(0)}W_3^{(1)} + \mathcal{A}F^{(0)}W_4^{(1)})]\right)\right]_y + \log \sum_{j=1}^C e^{z_j}$$

It's hideous but it's **one scalar function**. Backprop is just ∇_θ of this. The novelty for the 1-2-GNN is the **two parallel branches** inside the brackets — one node-level, one pair-level — joined by concatenation. They fan out forwards and merge backwards.

4 Backward pass

4.1 Logits $\rightarrow$ loss

$$\delta^z := \nabla_z \mathcal{L} = p - e_y \in \mathbb{R}^C$$

where e_y is the one-hot vector for the true class. Interpretation: increase the correct logit, decrease the wrong ones, in proportion to how much the model got it wrong.

4.2 Through the MLP

$$\begin{aligned} \nabla_{W_o} \mathcal{L} &= \delta^z b^\top, & \delta^b &= W_o^\top \delta^z, & \delta^a &= \delta^b \odot \mathbb{K}[a > 0] \\ \nabla_{W_c} \mathcal{L} &= \delta^a h_G^\top, & \delta^{h_G} &= W_c^\top \delta^a \end{aligned}$$

4.3 Split at the concatenation

The graph vector was built by concatenation, so the gradient splits by slicing:

$$\delta^{h_G^{(1)}} = \delta^{h_G} [1 : d_T], \quad \delta^{h_G^{(2)}} = \delta^{h_G} [d_T + 1 : 2d_T]$$

This is the key new step in the 1-2-GNN. One gradient signal becomes two, which then flow independently through the two branches until they meet again at $H^{(2)}$.

4.4 Pair branch: through the 2-GNN

Sum-pool back, ReLU mask, weight gradients (outer-product pattern), gradient w.r.t. $F^{(0)}$:

$$\begin{aligned} \delta^{F^{(1)}} &= \mathbf{1}_P (\delta^{h_G^{(2)}})^\top \\ \delta^{\mathcal{Z}^{(1)}} &= \delta^{F^{(1)}} \odot \mathbb{K}[\mathcal{Z}^{(1)} > 0] \\ \nabla_{W_3^{(1)}} \mathcal{L} &= (F^{(0)})^\top \delta^{\mathcal{Z}^{(1)}}, & \nabla_{W_4^{(1)}} \mathcal{L} &= (F^{(0)})^\top \mathcal{A} \delta^{\mathcal{Z}^{(1)}} \\ \delta^{F^{(0)}} &= \delta^{\mathcal{Z}^{(1)}} (W_3^{(1)})^\top + \mathcal{A} \delta^{\mathcal{Z}^{(1)}} (W_4^{(1)})^\top \end{aligned}$$

4.5 Through the bridge concatenation

Each row of $F^{(0)}$ was made of three pieces, so $\delta^{F^{(0)}}$ splits into three column-blocks:

$$\delta^{F^{(0)}} = [\delta^{\text{left}} \parallel \delta^{\text{right}} \parallel \delta^{\text{bond}}]$$

The first two route back to rows of $H^{(2)}$. Node u appears in $n - 1$ pairs and accumulates contributions from all of them:

$$\delta_{u,:}^{H^{(2)}, \text{from pairs}} = \sum_{v \neq u} \delta_{\{u,v\}}^{\text{block}(u,v)}$$

The bond gradient δ^{bond} is the gradient of $\mathcal{L}$ with respect to A_{uv} . **For classifier training A is fixed input data, so this gradient is computed but discarded.** *This is exactly the gradient that the dense wrapper preserves when we later train the generator* — flagged here, picked up in Part II.

4.6 Node branch and the merge at $H^{(2)}$

Node sum-pool back:

$$\delta_{v,:}^{H^{(2)}, \text{from nodes}} = \delta_G^{h^{(1)}} \quad \text{for every node } v$$

Merge. $H^{(2)}$ contributed to the loss through both branches, so the total gradient is the sum:

$$\delta^{H^{(2)}} = \delta^{H^{(2)}, \text{from nodes}} + \delta^{H^{(2)}, \text{from pairs}}$$

This is the structural reason the 1-GNN weights inside the 1-2-GNN learn different values from a standalone 1-GNN. Inside the hierarchical model, the node embeddings are pulled by *two* training pressures: “produce a good graph-level vector via sum-pool” and “produce good ingredients for pair-level computation.” These two pressures are summed at $H^{(2)}$ and propagated jointly.

4.7 Through the 1-GNN block

Same recipe as a standalone 1-GNN, starting from the merged $\delta^{H^{(2)}}$.

Layer 2:

$$\begin{aligned} \delta^{Z^{(2)}} &= \delta^{H^{(2)}} \odot \mathbb{K}[Z^{(2)} > 0] \\ \nabla_{W_1^{(2)}} \mathcal{L} &= (H^{(1)})^\top \delta^{Z^{(2)}}, \quad \nabla_{W_2^{(2)}} \mathcal{L} = (H^{(1)})^\top A \delta^{Z^{(2)}} \\ \delta^{H^{(1)}} &= \delta^{Z^{(2)}} (W_1^{(2)})^\top + A \delta^{Z^{(2)}} (W_2^{(2)})^\top \end{aligned}$$

Layer 1:

$$\begin{aligned} \delta^{Z^{(1)}} &= \delta^{H^{(1)}} \odot \mathbb{K}[Z^{(1)} > 0] \\ \nabla_{W_1^{(1)}} \mathcal{L} &= X^\top \delta^{Z^{(1)}}, \quad \nabla_{W_2^{(1)}} \mathcal{L} = X^\top A \delta^{Z^{(1)}} \end{aligned}$$

Done. Closed-form gradients for all eight weight matrices.

5 Weight update

$$W \leftarrow W - \eta \cdot \text{Adam}(\nabla_W \mathcal{L})$$

with $\eta = 0.01$. **Adam = momentum + per-parameter adaptive step sizes, more stable than plain gradient descent.** Not necessary, just a much better default.

6 What you’ve built

After training, you have a deterministic function

$$f_\theta : (A, X) \mapsto p \in \Delta^{C-1}$$

with θ frozen. Same input $\rightarrow$ same output. Dropout is off. The whole training apparatus only existed to find good θ .

For everything in Part II, this f_θ becomes the **frozen oracle** the generator is trying to satisfy. Its weights never change again. The dense wrapper (Part II section 7) re-implements its forward pass with dense matrix ops so that the *bond gradient* δ^{bond} — the one we just discarded — can flow through and reach the generator’s weights.

7 Two facts to memorize for the defence

1. **Every weight gradient is an outer product:** $(\text{input that the weight saw})^\top \times (\text{gradient of the weight's output})$. This is the universal pattern.
2. **The graph structure A (and $\mathcal{A}$ at the pair level) appears symmetrically in forward and backward passes.** Forward: who aggregates from whom. Backward: who routes gradients to whom.

And the one new fact specific to the 1-2-GNN:

The error signal at h_G **splits** at the concatenation, flows independently through the node branch and the pair branch, and **merges** back together at $H^{(2)}$ via summation. This is why the 1-GNN inside the hierarchical model learns different weights than a standalone 1-GNN.

Part II — The Generator: GIN-Graph Training Pipeline

A complete walk-through of how the GIN-Graph generator is trained for a single target class c , given a *frozen* pretrained classifier f_θ . Builds directly on Part I.

8 The big picture

What we’re trying to do. Reverse-engineer what the frozen classifier f_θ has internally learned about class c , by training a generator G_ϕ to produce graphs that the classifier confidently assigns to class c while remaining structurally realistic. The generated graphs are not “what real class- c graphs look like” — they’re “what f_θ thinks class- c graphs look like.” That distinction is the entire point.

Why a generator and not direct optimization. Optimizing a single graph to maximize $p_\theta(y = c)$ produces adversarial garbage that fools the classifier without looking real. Training a generator that has to fool a separate discriminator at the same time forces the outputs to lie near the real-data manifold, and gives you a *distribution* of explanations instead of a single point.

Three networks involved, only two are learning:

Network	Role	Status
Generator G_ϕ	Produces fake graphs from noise	Trainable
Discriminator D_ψ	Scores graphs as real/fake	Trainable
Frozen classifier f_θ	Provides class-targeting signal	Frozen forever

One generator per class. The class label is hard-coded into the loss via the target c in $\mathcal{L}_{\text{GNN}}$, not passed as an input to the generator.

9 Generator architecture

9.1 Stage 1: MLP backbone (Eqs 13–15)

$$h = \text{LeakyReLU}(W_2^G \text{LeakyReLU}(W_1^G z)) \in \mathbb{R}^{2d_h}$$

with $z \in \mathbb{R}^{d_z}$ standard Gaussian noise, $d_z = 32$, $d_h = 128$. Two output heads:

$$\tilde{A}_{\text{raw}} = \text{sym}(\text{reshape}(hW_A, N \times N))$$

$$\tilde{X}_{\text{raw}} = \text{reshape}(hW_X, N \times D)$$

where $\text{sym}(M) = \frac{1}{2}(M + M^\top)$ enforces undirected graphs. **These are real-valued logits, not yet a graph.**

9.2 Stage 2: Gumbel-Softmax sampling (Eqs 16–18)

The discrete-sampling problem: any naïve “sample an edge from a Bernoulli” or “argmax to pick an atom type” is piecewise constant in the logits and has zero gradient w.r.t. ϕ . Backprop dies.

The Gumbel trick. Adding Gumbel noise $g_i = -\log(-\log u_i)$ with $u_i \sim \text{Uniform}(0, 1)$ to logits π_i and taking the argmax produces samples with categorical probability $\frac{e^{\pi_i}}{\sum_j e^{\pi_j}}$. This separates the randomness (the g_i , drawn fresh and treated as fixed inputs) from the parameters.

Gumbel-Softmax. Replace argmax with softmax at temperature τ :

$$y_i = \frac{\exp((\pi_i + g_i)/\tau)}{\sum_{j=1}^K \exp((\pi_j + g_j)/\tau)}$$

As $\tau \rightarrow 0$ this recovers the discrete Gumbel sample. At $\tau = 1$ (training) it’s smooth and differentiable. At $\tau = 0.1$ (evaluation) it’s nearly discrete.

Straight-through estimator. Use a *hard* sample on the forward pass (so the classifier sees a real-looking graph) and the *soft* sample’s gradient on the backward pass. Implemented as $y_{\text{ST}} = y_{\text{hard}} + (y_{\text{soft}} - \text{stop_grad}(y_{\text{soft}}))$.

Both $\tilde{A}$ and $\tilde{X}$ go through Gumbel-Softmax. Both must be one-hot because the classifier was trained on discrete inputs.

10 The full generator loss

$$\mathcal{L}_G(\phi) = (1 - \lambda(t)) \mathcal{L}_{\text{GAN}}(\phi) + \lambda(t) \mathcal{L}_{\text{GNN}}(\phi) + \mathcal{L}_{\text{reg}}(\phi)$$

Three components, each pulling the generator in a different direction. Each one produces a gradient that arrives at $(\tilde{A}, \tilde{X})$ via a different upstream path; the three gradients sum at $(\tilde{A}, \tilde{X})$ and then share the same downstream path through Gumbel-Softmax and the MLP into ϕ .

The expectations are estimated by Monte Carlo: $B = 64$ fresh noise samples per training step, averaged. Unbiased estimator with variance σ^2/B — the finite-batch noise is exactly why Adam’s smoothing matters.

10.1 Where the class enters each component

Component	Class-specific?	How
$\mathcal{L}_{\text{GAN}}$	No	Discriminator sees all real graphs pooled across classes.
$\mathcal{L}_{\text{GNN}}$	Yes, directly	Target class c is the fixed argument of $-\log p_\theta(y = c \mid \cdot)$.
$\mathcal{L}_{\text{size}}(\text{reg})$	Yes, via summary stat	Uses $\bar{n}_c$, the mean node count of <i>class</i> c graphs.
$\mathcal{L}_{\text{type}}(\text{reg})$	Yes, via summary stat	Uses p_i^c , the atom-type frequencies of <i>class</i> c graphs.

Only $\mathcal{L}_{\text{GNN}}$ runs the classifier at training time.

11 Component 1: $\mathcal{L}_{\text{reg}}$ (structural regularization)

11.1 Size penalty

$$\mathcal{L}_{\text{size}} = \lambda_{\text{size}} \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2$$

with $\lambda_{\text{size}} = 10$. The trick is the **soft node count**:

$$\hat{n} = \sum_{v=1}^N \sigma(10(d_v - 0.5)), \quad d_v = \sum_{u=1}^N \tilde{A}_{vu}$$

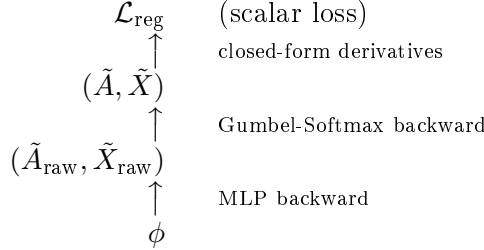
The factor of 10 makes the sigmoid steep, so it acts as an approximate step at $d_v = 0.5$ — “this slot is active iff it has more than half a neighbour’s worth of edges.”

11.2 Node-type penalty

$$\mathcal{L}_{\text{type}} = \lambda_{\text{type}} \sum_{i=1}^D (\hat{p}_i - p_i^c)^2, \quad \hat{p}_i = \frac{1}{N} \sum_{v=1}^N \tilde{X}_{v,i}$$

with $\lambda_{\text{type}} = 1$. Squared L2 distance between generated and real atom-frequency vectors.

11.3 Gradient path



No discriminator, no frozen classifier, no dense wrapper. **The cleanest path of the three.**

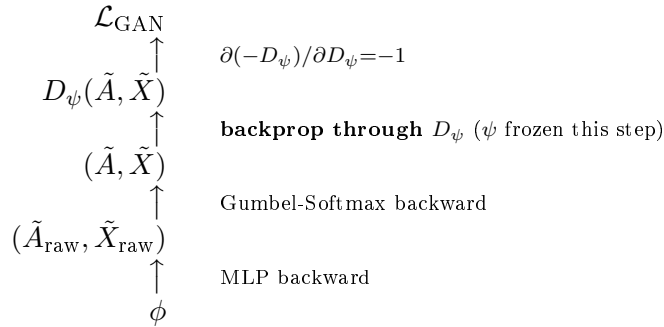
12 Component 2: $\mathcal{L}_{\text{GAN}}$ (adversarial loss)

The discriminator D_ψ is a small GNN that takes a graph and outputs a single real-valued score — high = looks real, low = looks fake.

$$\mathcal{L}_{\text{GAN}}(\phi) = -D_\psi(G_\phi(z))$$

averaged over the batch. Minimize $\rightarrow$ push score up $\rightarrow$ fool the discriminator.

12.1 Gradient path



The discriminator’s weights ψ are *not* updated by this gradient — they’re held fixed during the generator step. Updates to ψ happen in the alternating discriminator step, with a different loss.

Class-agnostic. Discriminator sees all real graphs pooled across classes.

WGAN-GP in one sentence: A more stable flavour of GAN training, the standard choice for graph GANs. The “GP” gradient penalty is a stability trick added to the *discriminator’s* loss; it doesn’t appear in the generator’s loss at all.

13 Component 3: $\mathcal{L}_{\text{GNN}}$ (classifier-guidance loss)

The term that does the actual explanation work. The only term that runs the frozen classifier at training time.

$$\mathcal{L}_{\text{GNN}}(\phi) = -\log p_{\theta}(y = c \mid \tilde{A}, \tilde{X})$$

Same cross-entropy formula as classifier training, but with target class c fixed and the gradient flowing into ϕ instead of θ .

13.1 The full chain rule

$$\nabla_{\phi} \mathcal{L}_{\text{GNN}} = \underbrace{(p - e_c)}_{(4) \text{ cross-entropy}} \cdot \underbrace{\frac{\partial z_{\text{logits}}}{\partial(\tilde{A}, \tilde{X})}}_{(3) \text{ DENSE WRAPPER backward through frozen } f_{\theta}} \cdot \underbrace{\frac{\partial(\tilde{A}, \tilde{X})}{\partial(\tilde{A}_{\text{raw}}, \tilde{X}_{\text{raw}})}}_{(2) \text{ Gumbel-Softmax backward}} \cdot \underbrace{\frac{\partial(\tilde{A}_{\text{raw}}, \tilde{X}_{\text{raw}})}{\partial \phi}}_{(1) \text{ generator MLP backward}}$$

13.2 Gradient path (visualised)

$$\begin{array}{c} \mathcal{L}_{\text{GNN}} = -\log p_{\theta}(y = c \mid \tilde{G}) \\ \uparrow \\ z_{\text{logits}} \in \mathbb{R}^C \quad (4) \ \delta^z = p - e_c \\ \uparrow \\ \text{MLP head of } f_{\theta} \quad (\text{frozen } W_c, W_o) \\ \uparrow \\ h_G = [h_G^{(1)} \parallel h_G^{(2)}] \\ \uparrow \quad \text{split at concat, sum-pool unfold} \\ H^{(T_1)}, F^{(T_2)} \\ \uparrow \\ (3) \text{ DENSE WRAPPER: backward through } f_{\theta} \\ \uparrow \quad \text{1-GNN block + 2-GNN block (frozen weights)} \\ \uparrow \quad \tilde{A} \text{ enters in 3 channels} \\ (\tilde{A}, \tilde{X}) \\ \uparrow \\ (2) \text{ Gumbel-Softmax backward} \\ (\tilde{A}_{\text{raw}}, \tilde{X}_{\text{raw}}) \\ \uparrow \\ (1) \text{ generator MLP backward} \\ \phi \end{array}$$

Factor (3) is the dominant cost — every generator step requires a full classifier forward+backward pass via the dense wrapper. The classifier’s weights θ get gradients computed for them by auto-grad, but the optimizer doesn’t include θ in its parameter list.

14 The dense wrapper (section 3.5) — *the technical contribution*

14.1 The problem

The pretrained classifier was implemented in PyTorch Geometric using **sparse representations**: graphs are stored as edge lists (a tensor of integer index pairs) and message passing is done via scatter-add operations on these lists. The classifier’s forward function literally has the signature `forward(x, edge_index)` — it does not accept an adjacency matrix at all.

To use this function on the generator’s continuous output $\tilde{A} \in [0, 1]^{N \times N}$, you would have to convert $\tilde{A}$ to an edge list first, which means *thresholding*. Thresholding is a step function:

zero gradient almost everywhere. After this conversion, the integer edge list has no autograd connection back to $\tilde{A}$. **Factor (3) of the chain rule is zero**, the entire product chain collapses, and the generator gets no signal from $\mathcal{L}_{\text{GNN}}$.

So the issue isn't "the gradient is bad." The issue is that $\tilde{A}$ literally cannot enter the original classifier as a continuous tensor.

14.2 The fix

The **dense wrapper** is a re-implementation of the same classifier with a different forward function. Same frozen weights, copied from the pretrained model. Same mathematical function on discrete inputs. But the forward function uses **dense matrix operations** ($\tilde{A}HW$ via `torch.matmul`) instead of sparse scatter, and accepts $\tilde{A}$ as a continuous tensor directly. Now $\tilde{A}$ enters as a `torch.Tensor` of dtype `float32` and gets used in differentiable operations, autograd records every step on its computation graph, and on `.backward()` it produces a well-defined $\partial z_{\text{logits}}/\partial \tilde{A}$.

It's not a different model. Same weights, same function, different code path.

14.3 Dense 1-GNN forward (Eq 25)

$$H^{(t)} = \sigma(H^{(t-1)}W_1^{(t)} + \tilde{A}H^{(t-1)}W_2^{(t)})$$

The line $\tilde{A}H^{(t-1)}W_2^{(t)}$ is the only thing that changes from the original. With $T_1 = 3$ layers, $\tilde{A}$ enters in 3 matmul operations.

14.4 Dense 1-2-GNN — the actual contribution

Dense wrappers for plain 1-GNNs are standard. **The contribution of this thesis is extending the technique to the hierarchical 1-2-GNN**, which requires three additional considerations.

Dense pair feature construction (Eq 26). For every pair of slots (i, j) :

$$F[i, j] = [H_{\min(i, j), :}^{(T_1)} \parallel H_{\max(i, j), :}^{(T_1)} \parallel \tilde{A}_{ij}]$$

The bond bit $\tilde{A}_{ij}$ is the **continuous** value taken straight from the soft adjacency. Second channel.

Dense pair-neighbour aggregation (Eqs 27–28). The swap-one-element neighbour rule becomes a **soft attention sum** weighted by $\tilde{A}$:

$$\begin{aligned} \text{agg}_1[i, j] &= \sum_{\substack{w=1 \\ w \neq j}}^N \tilde{A}_{iw} \cdot g[j, w] \\ \text{agg}_2[i, j] &= \sum_{\substack{w=1 \\ w \neq i}}^N \tilde{A}_{jw} \cdot g[i, w] \end{aligned}$$

where $g = FW_4^{(\ell)}$ is the transformed pair feature tensor. Each candidate w contributes to the aggregation in proportion to how strongly $\tilde{A}_{iw}$ says it's a neighbour — soft attention recovering the discrete swap-rule in the limit.

The constraints $w \neq j$ and $w \neq i$ are essential **boundary masking**: without them, the sums include the degenerate self-pair $\{j, j\}$ or $\{i, i\}$. Enforced by zeroing the diagonal of g before the einsum.

Dense 2-GNN layer update (Eq 29):

$$F^{(\ell+1)} = \sigma(F^{(\ell)}W_3^{(\ell)} + \text{agg}_1^{(\ell)} + \text{agg}_2^{(\ell)})$$

14.5 Where $\tilde{A}$ enters the autograd graph in the 1-2-GNN dense wrapper

Three distinct channels:

1. **1-GNN matmuls:** $\tilde{A}H^{(t-1)}$ in each of $T_1 = 3$ message-passing layers.
2. **Pair bond bits:** $\tilde{A}_{ij}$ embedded in the pair feature for every pair.
3. **Soft attention in pair aggregation:** $\tilde{A}_{iw}$ as the weight in agg_1 and $\tilde{A}_{jw}$ in agg_2 , in each of $T_2 = 2$ 2-GNN layers.

On the backward pass, autograd accumulates gradient contributions from **all three channels** into $\partial z_{\text{logits}}/\partial \tilde{A}$. This three-channel gradient flow is the technical reason the 1-2-GNN dense wrapper is the contribution: nobody had handled the second and third channels before for k -GNN explanation generation.

14.6 Defence answer in one paragraph

The pretrained classifier uses sparse PyTorch Geometric operations whose forward function accepts only edge lists, not continuous adjacency matrices. To compute $\nabla_{\phi} \mathcal{L}_{\text{GNN}}$, the chain rule needs $\partial z_{\text{logits}}/\partial \tilde{A}$, which doesn't exist in the sparse implementation. The dense wrapper re-implements the classifier's forward pass using dense matmul operations — same frozen weights, same mathematical function on discrete inputs — so that $\tilde{A}$ enters as a continuous tensor in differentiable operations. Dense wrappers for 1-GNNs are standard; my contribution is extending the technique to the hierarchical 1-2-GNN, which required correctly translating the swap-based discrete pair-neighbourhood into a soft-attention dense computation while preserving gradient flow through both the bond bits and the pair-aggregation weights.

15 Dynamic weight $\lambda(t)$ (curriculum learning, Eq 23)

$\lambda(t) \in [0, 1]$ controls the trade-off between $\mathcal{L}_{\text{GAN}}$ and $\mathcal{L}_{\text{GNN}}$. Scheduled hyperparameter, not learnable.

$$\lambda(t) = \sigma\left(k \cdot \left(\frac{2(t/T - p)}{1 - p} - 1\right)\right)$$

with T = total iterations, $p = 0.4$, $k = 10$. $\lambda \approx 0$ for the first 40% (realism phase), rises sharply, saturates at $\lambda \approx 1$ (class-targeting phase). **Curriculum:** realism first, class targeting later. Treated as a constant scalar at each step.

16 The full training loop

16.1 Step 1: Update discriminator ψ

$$\mathcal{L}_D = \frac{1}{B} \sum_i D_{\psi}(\tilde{G}_i) - \frac{1}{B} \sum_i D_{\psi}(G_i) + \lambda_{\text{GP}} \cdot (\text{gradient penalty})$$

Compute $\nabla_{\psi} \mathcal{L}_D$, take Adam step on ψ . ϕ unchanged.

16.2 Step 2: Update generator ϕ

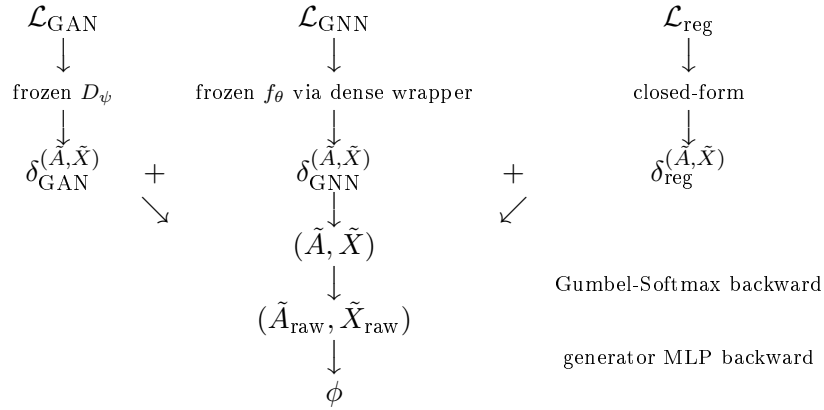
$$\mathcal{L}_G = (1 - \lambda(t)) \left(-\frac{1}{B} \sum_i D_\psi(\tilde{G}_i) \right) + \lambda(t) \left(-\frac{1}{B} \sum_i \log p_\theta(y = c \mid \tilde{G}_i) \right) + \mathcal{L}_{\text{reg}}$$

Compute the full gradient:

$$\nabla_\phi \mathcal{L}_G = (1 - \lambda(t)) \nabla_\phi \mathcal{L}_{\text{GAN}} + \lambda(t) \nabla_\phi \mathcal{L}_{\text{GNN}} + \nabla_\phi \mathcal{L}_{\text{reg}}$$

16.3 The merged gradient picture

Three loss components, three upstream paths, **one merge at $(\tilde{A}, \tilde{X})$** , one shared downstream path:



Same split-and-merge structure as the 1-2-GNN classifier backward pass, but at a higher level: there, the merge was between node-level and pair-level branches inside one network. Here, the merge is between three independent loss components.

Take Adam step on ϕ . ψ and θ unchanged.

16.4 Repeat for ~ 300 epochs

After training, G_ϕ is a deterministic function from noise to a class- c explanation graph.

17 After training: generating explanations

1. **Sample noise.** Draw $z_1, \dots, z_{100} \sim \mathcal{N}(0, I)$. Class is implicit in *which* generator you use.
2. **Generate.** Push each z_i through MLP and Gumbel-Softmax. Evaluation temperature $\tau = 0.1$ for sharper outputs.
3. **Score.** Each generated graph gets (s, p, d) , combined into $v = (s \cdot p \cdot d)^{1/3}$. (Part III.)
4. **Analyze.** Validity rates, atom-type distributions, cross-model classification.

Part III — The Validation Score

After all training is done, both networks are black boxes. Pure evaluation, no training, no gradients.

18 The composite score (Eq 31)

$$v = (s \cdot p \cdot d)^{1/3}$$

A **geometric mean** of three components, each in $[0, 1]$. A graph is “valid” if $v > 0.5$ and its average degree is within 3 standard deviations of the class mean.

Why geometric mean and not arithmetic? With the geometric mean, if any single component is near zero, v is near zero — no compensation. A graph with $s = 0.95, p = 0.95, d = 0.05$ scores ≈ 0.36 , correctly flagged as bad. An arithmetic mean would let it score ≈ 0.65 , hiding the connectivity problem.

19 Component 1: prediction probability p

$$p = p_\theta(y = c \mid \tilde{G})$$

The frozen classifier’s softmax probability for the target class. **What it measures:** “Does the classifier confidently call this graph class c ?” Almost always near 1.0 in your results — the generator was trained to maximize this. Sanity check more than discriminating metric.

20 Component 2: embedding similarity s — *the corrected one*

$$s = \cos(h_{\tilde{G}}, \mu_c) = \frac{h_{\tilde{G}} \cdot \mu_c}{\|h_{\tilde{G}}\| \|\mu_c\|}$$

where:

- $h_{\tilde{G}}$ is the **frozen classifier’s internal graph embedding** of the generated graph, accessed via the dense wrapper’s `get_embedding()` method.
- μ_c is the **class centroid** — the mean embedding of all real class- c graphs in the training set, computed once and stored:

$$\mu_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} h_G$$

What it measures: “Does the generated graph live in the same region of the classifier’s internal representation space as real class- c graphs?” High s = “the classifier sees this graph the same way it sees real class- c examples.” Low s = “classifier classifies it correctly via a weird path through embedding space.”

This is where **fidelity** lives. A graph can have $p = 1.0$ but $s = 0.15$ — adversarial-style outcome. PROTEINS Enzyme: 1-GNN gets $s = 0.356$ and 1-2-GNN gets $s = 0.151$ even though both have $p \geq 0.92$.

20.1 The “corrected” part — second contribution of the thesis

The original GIN-Graph implementation **used the prediction probability p as a proxy for embedding similarity**, effectively setting $s = p$. So the original score was:

$$v_{\text{original}} = (p \cdot p \cdot d)^{1/3} = (p^2 \cdot d)^{1/3}$$

Wrong, because p and “similarity to class centroid” measure genuinely different things. A graph can score perfectly on p while being totally atypical in embedding space.

Your fix: actually compute the cosine similarity between the generated graph’s embedding (via the dense wrapper) and a precomputed class centroid. This gives a faithful measurement of how similar the generated graph is to real class members in the model’s own learned space.

Defence answer: “The original GIN-Graph paper conflated embedding similarity with prediction probability, using p in place of s . I corrected this by computing actual cosine similarity between the generated graph’s embedding and a stored class centroid. This matters because the two quantities can disagree — a graph can be confidently classified into the target class while having an embedding far from typical class members, indicating an adversarial-style explanation rather than a faithful one.”

21 Component 3: degree score d

$$d = \exp\left(-\frac{(\bar{d}_{\tilde{G}} - \mu_d)^2}{2\sigma_d^2}\right)$$

where $\bar{d}_{\tilde{G}} = 2|\tilde{E}|/|\tilde{V}|$ is the average degree of the generated graph and μ_d, σ_d are the mean and std of average degrees across real class- c graphs.

A **Gaussian kernel** centered at the class mean. $\bar{d}_{\tilde{G}} = \mu_d$ exactly $\Rightarrow d = 1$. One std $\Rightarrow d \approx 0.61$. Three std $\Rightarrow d \approx 0.011$.

What it measures: coarse structural realism via average degree. Can’t catch finer issues (uneven distributions, wrong motif frequencies).

22 Granularity κ (Eq 34, separate from v)

$$\kappa = 1 - \min\left(1, \frac{n_{\tilde{G}}}{\bar{n}_c}\right)$$

What it measures: how *small* the explanation is relative to a typical class graph. Values near 0 = “explanation is graph-sized” (coarse).

In your results, $\kappa \approx 0$ everywhere — structural property of GIN-Graph. The generator outputs fixed-size $N \times N$ tensors and the size regularizer pushes $\hat{n}$ toward $\bar{n}_c$, so by construction explanations are graph-sized. Section 6.3 of the thesis discusses this honestly: it’s a limitation of the model-level approach, not a bug. Model-level methods inherently answer “what does a typical class member look like?” rather than “which subgraph drives the prediction?”

23 How scores get used in section 5

- **Validity rate** — fraction of graphs with $v > 0.5$ and degree within 3 std. Headline number per configuration in Tables 4 and 5.

- **Mean validation score** — average v over 100 generated graphs.
- **Component decomposition** — separately report s, p, d in Table 6 to show which one drags the score down. This is how PROTEINS Enzyme’s “high p , low s ” pattern is discovered.
- **Top-ranked explanations** — sort by v , show the highest in figures.

All computed with both networks frozen. No training, no gradients.

Defence Prep Checklist

24 The two technical contributions

1. **Dense wrapper for the hierarchical 1-2-GNN.** Re-implements the frozen classifier with dense matmul ops so $\tilde{A}$ can enter as a continuous differentiable tensor. The 1-GNN dense wrapper is standard; extending to 1-2-GNN required handling three channels of $\tilde{A}$ entry (matmuls, bond bits, soft-attention pair aggregation) plus the boundary masking that excludes degenerate self-pairs.
2. **Corrected validation metric.** Original GIN-Graph used p in place of s , double-counting prediction probability. The fix computes actual cosine similarity to a class centroid via the dense wrapper’s embedding access, exposing cases where high-confidence classifications correspond to atypical embeddings.

25 The two structural facts to internalize

Classifier backward pass: error signal at h_G **splits** at the concatenation, flows independently through node and pair branches, and **merges** at $H^{(2)}$ via summation. This is why the 1-GNN inside the 1-2-GNN learns different weights than a standalone 1-GNN.

Generator backward pass: three loss components, three different upstream gradient sources at $(\tilde{A}, \tilde{X})$ — frozen D_ψ , frozen f_θ via dense wrapper, closed-form. The three gradients **sum** at $(\tilde{A}, \tilde{X})$, then share the same downstream path through Gumbel-Softmax and the MLP into ϕ . Same split-and-merge pattern as the classifier, one level higher.

26 The clean defence one-liners

“Why does GIN-Graph need Gumbel-Softmax?” Discrete sampling has zero gradient w.r.t. parameters, so backprop from the classifier loss can’t reach the generator. Gumbel-Softmax is a continuous relaxation of discrete sampling that’s differentiable in the logits. Straight-through gives discrete forward pass + soft backward pass.

“Why does GIN-Graph need the dense wrapper?” The original sparse classifier’s forward function accepts only edge lists, not continuous adjacency matrices, so $\partial z_{\text{logits}} / \partial \tilde{A}$ doesn’t exist. The dense wrapper re-implements the same function with dense matmul ops, accepting $\tilde{A}$ as a continuous tensor.

“Why is the dense wrapper your contribution and not just standard?” Dense wrappers for plain 1-GNNs are standard. The hierarchical 1-2-GNN dense wrapper requires translating the swap-based discrete pair-neighbourhood into a soft-attention dense computation, with three distinct channels of $\tilde{A}$ entry and boundary masking to exclude degenerate self-pairs.

“Why train one generator per class instead of a conditional generator?” Cleaner experimental comparison, stronger per-class signal, no conditioning artifacts. The class is hard-coded into $\mathcal{L}_{\text{GNN}}$ during training, baked into ϕ , and the trained generator is single-purpose.

“What does $\lambda(t)$ do?” Curriculum learning — generator focuses on realism first ($\lambda \approx 0$), then on class-targeting ($\lambda \approx 1$). Without this, the generator gets pulled in two directions from epoch 1 and tends to collapse.